

Noise-Identifikation durch das Heuristics-Miner-Verfahren selbst

Im Gegensatz zur vorherigen Analyse ([noise_analysis.md](#)), die Noise "von außen" über seltene Trace-Varianten identifiziert hat, wird hier der **interne Noise-Filter des Heuristics-Miner-Algorithmus** selbst ausgewertet: pm4py filtert vor der eigentlichen Abhängigkeitsberechnung automatisch Kanten aus dem Directly-Follows-Graphen (DFG) heraus, die als Rauschen gelten.

Der Mechanismus des Heuristics Miners

Der Algorithmus arbeitet in zwei Stufen mit zwei Schwellenwerten (pm4py-Standardwerte):

Parameter	Standardwert	Bedeutung
dfg_pre_cleaning_noise_thresh	0,05	Kanten des DFG, deren Häufigkeit unter 5 % der stärksten Ausgangskante derselben Aktivität liegt, werden vor der Modellbildung als Rauschen verworfen.
dependency_thresh	0,5	Nur Kanten, die diese Vorreinigung überleben, werden auf einen Abhängigkeits-("Kausalitäts")-Wert geprüft; erst ab 0,5 gelten sie als echte Prozessabhängigkeit.

Ergebnis

Aus dem rohen, ungefilterten Directly-Follows-Graphen des Logs mit **55 Kanten** verwirft der Heuristics Miner bereits in der Vorreinigungsstufe **28 Kanten (50,9 %)** als Rauschen – **bevor** überhaupt der Abhängigkeitswert berechnet wird. Von den verbleibenden 27 Kanten übersteht **jede einzelne** locker den Abhängigkeits-Schwellenwert (Werte zwischen 0,98 und 0,999) – der Abhängigkeits-Schwellenwert filtert hier also gar nichts zusätzlich heraus. **Die gesamte Noise-Erkennung des Verfahrens erfolgt faktisch allein über den Vorreinigungs-Schwellenwert von 5 %.**

Die 28 vom Algorithmus als Noise verworfenen Kanten (nach Häufigkeit sortiert)

Kante	Log-Häufigkeit
Receive Confirmation → Pick & Pack Items	9
Process Payment → Ship Order	7
Pick & Pack Items → Send Tracking Information	6
Browse & Select Products → Validate Order & Check Stock	6
Validate Order & Check Stock → Process Payment	6
Receive Tracking Information → Validate Order & Check Stock	4

Kante	Log-Häufigkeit
Receive Tracking Information → Browse & Select Products	4
Ship Order → Receive Tracking Information	4
Receive Tracking Information → Receive Confirmation	4
Place Order → Browse & Select Products	3
Ship Order → Pick & Pack Items	3
Process Payment → Receive Confirmation	3
Receive Tracking Information → Ship Order	3
Place Order → Receive Confirmation	3
Receive Tracking Information → Place Order	3
Pick & Pack Items → Process Payment	2
Approve Bulk Order → Process Payment	2
Process Payment → Express Ship Order	2
Send Tracking Information → Ship Order	2
Approve Bulk Order → Validate Order & Check Stock	2
Receive Tracking Information → Pick & Pack Items	2
Place Order → Approve Bulk Order	2
Receive Tracking Information → Send Tracking Information	2
Receive Confirmation → Validate Order & Check Stock	1
Validate Order & Check Stock → Place Order	1
Express Ship Order → Receive Tracking Information	1
Receive Tracking Information → Process Payment	1
Receive Tracking Information → Express Ship Order	1

Abgleich mit der manuellen Noise-Analyse

Diese 28 algorithmisch verworfenen Kanten stimmen **zu 100 % (28/28)** mit den zuvor manuell identifizierten "noise-only" Kanten überein, die ausschließlich in den 64 künstlich verrauschten Traces auftraten (siehe [noise_analysis.md](#)). Das bestätigt unabhängig: Der Heuristics Miner erkennt exakt das synthetisch eingefügte Rauschen (fehlende, doppelte und vertauschte Events) und filtert es korrekt heraus.

Eine bemerkenswerte Abweichung: Die Selbstschleife [Receive Tracking Information → Receive Tracking Information](#) (Häufigkeit 4, resultiert aus doppelten Events in Noise-Traces) wird von der manuellen Varianten-Analyse ebenfalls als "nur in Noise-Traces vorkommend" eingestuft – **übersteht aber die algorithmische Vorreinigung** (Abhängigkeitswert 0,9988) und bleibt Teil des finalen

Heuristics-Netzes. Das erklärt auch, warum diese spezielle Relation in der vorherigen Konformitätsprüfung **nicht** unter den 25 Footprint-Abweichungen auftauchte: Der Algorithmus lässt in diesem einen Fall (Selbstschleifen werden über einen eigenen Mechanismus bewertet, nicht über die relative 5-%-Schwelle regulärer Kanten) fälschlicherweise Rauschen als scheinbar echtes Verhalten durch.

Fazit

Das Heuristics-Miner-Verfahren identifiziert Rauschen nicht durch Betrachtung einzelner Traces, sondern **aggregiert auf Ebene der Directly-Follows-Kanten**: Jede Kante, die seltener als 5 % der dominanten Ausgangskante derselben Quellaktivität auftritt, wird verworfen. Im vorliegenden Log trifft das auf 28 von 55 Kanten (50,9 %) zu – und diese Kanten sind (bis auf eine Selbstschleifen-Ausnahme) exakt die Kanten, die durch die 64 künstlich verrauschten Traces (5,82 % des Logs) erzeugt wurden. Der 5-%-Schwellenwert des Verfahrens erweist sich damit für dieses Log als sehr treffsicherer, weitgehend automatischer Noise-Filter.

Artefakte

- `heuristic_miner_noise.py` — Skript (vergleicht rohen DFG mit dem intern gefilterten DFG des Heuristics Miners)
- `heuristic_noise_report.txt` — vollständiger Rohbericht